

Spinning Up Modern: Technical Modernization Report

Kiet Huynh

April 2026

1 Core Architectural Evolution

This report details the technical implementation of the modernization of OpenAI’s Spinning Up repository. The primary goal was to transition from legacy, unoptimized software to a production-grade, GPU-accelerated research environment using the latest available library versions.

1.1 Gymnasium Migration and Environment Standardization

The project successfully transitioned from `gym` to `gymnasium` (v1.0+).

Table 1: API and Environment Standardization

Feature	Implementation Detail	Version/API
Step Logic	Decomposed Terminal/Truncated signals	<code>obs, rew, term, trunc, info</code>
Reset Logic	Information dictionary support	<code>obs, info</code>
CartPole	Standardized on stable version	<code>CartPole-v1</code>
HalfCheetah	Standardized on MuJoCo free release	<code>HalfCheetah-v5</code>

2 Deep Learning Backends

2.1 PyTorch 2.5+: Graph Mode and GPU Coordination

PyTorch implementations have been optimized for modern hardware and software standards:

- **Kernel Fusion:** Integrated `torch.compile()` for JIT-optimized execution.
- **Memory Management:** Fixed host-device transfer bottlenecks by standardizing on `.detach().cpu().numpy()` for all environment interactions.

- **Type Safety:** Implemented comprehensive Python type hints for all Actor-Critic architectures.

2.2 TensorFlow 2.15: Eager Execution Migration

A total rewrite of the TensorFlow backend removed all legacy `tf.compat.v1` dependencies.

- **Functional Models:** All algorithms now use `tf.keras.Model` subclassing for better modularity.
- **Automatic Differentiation:** Replaced static graphs with `tf.GradientTape` orchestration.

```

1 with tf.GradientTape() as tape:
2     loss_pi, kl, ent = compute_loss_pi(obs, act, adv, logp_old)
3     grads = tape.gradient(loss_pi, ac.pi.trainable_variables)
4     pi_optimizer.apply_gradients(zip(grads, ac.pi.trainable_variables))

```

Listing 1: Modern TF2 Policy Update Pattern

3 Hardware Acceleration: Dynamic Library Resolution

A critical technical achievement was solving the **cuDNN Version Conflict** present in modern pip-based NVIDIA distributions.

3.1 Implementation Detail: Library Mapping

TensorFlow 2.15 expects `libcudnn.so.8`, while modern `nvidia-*` packages provide `v9`. We implemented a dynamic initialization hook in `setup_tf_gpu()`:

1. **Automated Discovery:** Recursive scanning of `site-packages` for valid NVIDIA shared objects.
2. **Symbolic Resolution:** Real-time creation of versioned symlinks (e.g., `v9` $\rightarrow$ `v8`) in a localized runtime directory.
3. **LD_PRELOAD Injection:** Dynamic environment patching before the first backend import to ensure 100% GPU detection.

4 Distributed Execution: MPI-GPU Synchronization

Modernized MPI support ensures correct coordination with CUDA devices:

- **Weight Sync:** Synchronized model parameters across processes using rank-0 broadcasting.
- **Gradient Aggregation:** Global gradient averaging via `mpi_avg` before optimizer updates.
- **Resource Isolation:** Configured `allow_growth` to prevent MPI ranks from conflicting on VRAM allocation.

5 Verification Results

The system passed a rigorous 100% sweep of the core algorithm suite (VPG, PPO, DDPG, TD3, SAC, TRPO) with no initialization warnings and full hardware acceleration.

6 Citation

```

1 @misc{SpinningUpModern2026,
2   author = {Kiet Huynh},
3   title = {{Spinning Up in Deep RL (Modernized)}},
4   year = {2026},
5   publisher = {GitHub},
6   howpublished = {\url{https://github.com/tkiethuynh/spinningup}}
7 }
```